

# PRISMA

## O que foi arrumado antes de o site ir para o ar

Este documento conta, bem devagar e sem palavra difícil, tudo o que foi verificado e consertado no site. Cada parte começa com uma comparação do dia a dia, para você entender *por que* aquilo importava — e só depois diz o que foi feito.

6 de setembro de 2026
19 itens verificados · 0 problemas restantes no que dependia do código

## 0 Primeiro: o que é isso tudo?

O **PRISMA** é um site com três portas. Pela primeira entram os **alunos**, que montam o currículo. Pela segunda entram as **empresas**, que publicam vagas. Pela terceira entra o **gestor**, que acompanha tudo.

> **IMAGINE ASSIM**
>
> Colocar um site no ar é como abrir uma loja. Antes de destrancar a porta, alguém precisa dar uma volta e conferir: a placa está com o nome certo? A campainha funciona? Se um cliente entrar na sala errada, alguém avisa ou ele fica perdido? A porta tem tranca?
>
> A sua lista de 19 itens é justamente essa volta de conferência. Foi ela que eu percorri, item por item.

E eu não fui só olhando o código. Eu **liguei o site de verdade**, abri num navegador de mentirinha que finge ser um celular, preenchi os formulários, apertei as teclas e cliquei nos links — igual a uma pessoa faria. Foi assim que os problemas apareceram.

## 1 O problema mais grave: o endereço errado

> **IMAGINE ASSIM**
>
> Você faz um convite de festa e, no lugar do endereço, escreve "*é na minha casa*". Para você faz todo sentido. Mas quem recebe o convite lê "minha casa" e vai para a **casa dela mesma** — e lá não tem festa nenhuma.

Era exatamente isso que estava escrito na tela de cadastro. Para provar que não é um robô, a pessoa precisa digitar um código que aparece numa imagenzinha embaçada (o *captcha*). O endereço dessa imagem estava anotado como `127.0.0.1`, que em linguagem de computador significa exatamente **"aqui, nesta máquina"**.

**O QUE IA ACONTECER**

No computador de quem programou, funcionava — porque o site estava ali mesmo. Mas no celular de um aluno, "aqui, nesta máquina" é o *celular dele*. A imagem nunca ia carregar.

Pior: sites de verdade usam cadeado (o *https*). O endereço estava escrito sem cadeado. O navegador tem uma regra de segurança que diz: "página com cadeado não pode buscar coisa sem cadeado". Ele ia **bloquear o pedido** antes mesmo de tentar.

Resultado: **ninguém conseguiria criar uma conta.** E o pior é que a tela não mostraria erro nenhum — só ficaria ali, sem funcionar, sem ninguém entender por quê.

**O que eu fiz:** em vez de deixar o endereço escrito à mão, o site agora *pergunta* qual é o endereço certo, uma única vez, e usa a resposta em todo lugar. É a diferença entre escrever "minha casa" no convite e escrever a rua e o número de verdade.

## 2 O site no celular

**IMAGINE ASSIM**

Você faz um desenho numa folha grande e depois tenta colar essa folha dentro de um caderninho pequeno. Ela não cabe: sobra desenho para fora, e você tem que arrastar a folha para os lados para ver o resto.

Era isso que acontecia. Algumas partes do site tinham um tamanho **fixo**, cravado no código — como uma folha que não sabe encolher. Num celular comum a tela é mais estreita que essa folha, então o site abria torto, com uma barra para arrastar de lado.

Encontrei quatro lugares assim e arrumei todos:

- ✓ **A primeira tela do site** — a pior de todas, porque é a primeira coisa que alguém vê. A caixa verde tinha tamanho fixo e o arquivo não tinha *nenhuma* regra para telas pequenas.
- ✓ **A tela de login** — a caixa tinha altura fixa. Quando aparecia uma mensagem de erro, o texto *transbordava* para fora da caixa, como água em copo cheio. E os três links de baixo ficavam colados: lia-se "CadastrarEsqueci minha senha", tudo junto.
- ✓ **A tela de cadastro** — o botão de trocar a imagem do captcha ficava para fora da tela. Existia, mas ninguém alcançava.
- ✓ **A prévia do currículo** — estava do tamanho exato de uma folha de papel A4. Faz sentido para *imprimir*, mas não para ver no celular. Agora ela encolhe na tela e volta ao tamanho de folha só quando vai para a impressora.

**COMO SEI QUE FUNCIONOU**

Abri as 8 telas públicas numa tela de **360 pontos de largura** (o tamanho de um celular bem comum) e medi quanto o conteúdo passava da borda. Antes passava. Agora o excesso é de **zero** em todas as 8.

## 3 Os formulários: a campainha sem fio

**IMAGINE ASSIM**

Você instala uma campainha bonita na porta. Ela tem botão, tem luzinha, parece perfeita. Mas ninguém ligou o fio dela na casa. Quem aperta o botão acha que chamou — e lá dentro não toca nada.

Nas telas de entrar e de se cadastrar, os campos de digitar estavam **soltos**. Em páginas de internet existe uma "caixinha especial" que agrupa campos e diz ao navegador: "*isto aqui é um formulário*". Essa caixinha não estava lá em **nenhuma** das 6 telas. Isso causava três coisas:

- **Apertar Enter não fazia nada.** A pessoa digitava a senha, apertava Enter — porque é o que todo mundo faz — e ficava esperando.
- **No celular, o teclado não ajudava.** Quando o navegador reconhece um formulário, ele troca a tecla Enter por um botão de "confirmar". Sem a caixinha, essa tecla nunca aparecia.
- **O navegador não oferecia guardar a senha.** Ele não entendia que aquilo era um login, então nunca perguntava "quer que eu lembre sua senha?". E no cadastro, nunca oferecia criar uma senha forte.

**O que eu fiz:** coloquei a caixinha nas 6 telas e avisei o navegador o que é cada campo — "este é o e-mail", "esta é a senha atual", "esta é uma senha nova", "este é um código que chegou por e-mail". Nos campos de código, o celular agora abre direto o **teclado de números**, em vez de o de letras.

**COMO SEI QUE FUNCIONOU**

Preenchi o login no navegador e apertei **só o Enter**, sem clicar em botão nenhum. O pedido saiu para o servidor, e a mensagem "Email ou senha inválidos." apareceu na tela — que é exatamente o que tinha que acontecer.

### De brinde: os rótulos que não apontavam para nada

Achei outra coisa no caminho. Em cima de cada campo tem uma palavrinha: "Email", "Senha". Essa palavrinha deveria estar *amarrada* ao campo, de forma que clicar nela já coloque o cursor lá dentro. A amarração estava quebrada. Duas consequências: clicar em "Email" não fazia nada, e — mais sério

— o programa que lê a tela em voz alta para pessoas cegas anunciava "campo de texto", sem dizer *qual* campo. Arrumado.

## 4 Quando a pessoa se perde: a página 404

> **IMAGINE ASSIM**
>
> Você entra num prédio procurando a sala 12. A sala 12 não existe. Aí um funcionário, **sem falar nada**, pega você pelo braço e te coloca de volta na portaria. Você não sabe se errou o número, se a sala mudou, ou se o prédio é o errado. Você só sabe que "não deu".

Era assim que o site tratava endereço errado: mandava a pessoa de volta para a primeira tela, calado. E o caso mais provável de isso acontecer é o mais chato: o link de confirmação de cadastro chegando quebrado no e-mail.

Tinha um segundo problema, invisível. O Google, ao visitar qualquer endereço errado, recebia a primeira tela e um recado de "está tudo certo aqui". Ele então achava que existiam dezenas de páginas iguais no site. Isso atrapalha o site a aparecer nas buscas.

**O que eu fiz:** criei uma tela de verdade. Ela mostra um "404" grande, diz *qual* endereço não existe, e oferece as saídas certas: voltar ao início, entrar na conta, ou — para o caso do link quebrado — pedir um código novo. E ela **não** troca o endereço na barra do navegador, para a pessoa poder ver o que digitou errado.

## 5 A velocidade: a caixa de brinquedos inteira

> **IMAGINE ASSIM**
>
> Uma criança quer brincar com um carrinho. Mas, para chegar ao carrinho, ela tem que carregar a **caixa de brinquedos inteira** da sala até o quarto — com a bola, o quebra-cabeça, os bonecos, tudo. Só depois de arrastar tudo é que ela pega o carrinho.

O site fazia isso. Quem abria só a tela de *login* baixava, junto e antes de ver qualquer coisa, as telas de aluno, de empresa e de gestor — e dentro delas um programa pesado que desenha **gráficos**. Só o gestor vê gráfico. Um aluno nunca vê nenhum. E mesmo assim baixava.

**O que eu fiz:** separei os brinquedos em caixinhas. Agora cada tela só é baixada no momento em que a pessoa realmente vai até ela. A tela de início e o login continuam vindo na hora — porque são a porta de entrada, e atrasá-los faria a espera piorar, não melhorar.

ANTES

**231 kB**

baixados só para ver a tela de login

DEPOIS

**95 kB**

59% menos coisa para carregar

Achei ainda um detalhe escondido. Uma peça grande e que *nunca muda* (a base do site) estava misturada com a parte que muda a cada atualização. Como estavam juntas, toda atualização obrigava o visitante a baixar as duas de novo. Separei: a parte que muda caiu de **200 kB para 22 kB**. Quem já visitou o site uma vez sente isso na segunda visita.

## 6 A cara do site: nome, foto e crachá

### O crachá emprestado (favicon)

> IMAGINE ASSIM
>
> É aquele desenhinho minúsculo que aparece na abinha do navegador, do lado do nome do site. É o crachá dele. O do PRISMA estava com o crachá de **outra empresa** — o logo da ferramenta que os programadores usaram para construir o site, que veio de fábrica e ninguém trocou.

Desenhei um crachá do PRISMA: um prisma de vidro em triângulo, com um raio de luz branco entrando de um lado e saindo do outro separado em cores. É a ideia do nome. Ele também aparece se alguém salvar o site na tela inicial do celular.

### O presente sem papel de embrulho (imagem de compartilhamento)

> IMAGINE ASSIM
>
> Quando você manda um link no WhatsApp, geralmente aparece um quadradinho bonito com foto e título. Esse quadradinho não é mágica: o site tem que *preparar* a foto e deixar avisado. O PRISMA não tinha nada preparado — o link chegava como **texto cru**, sem foto e sem título, com cara de coisa suspeita.

Criei essa imagem no tamanho exato que o WhatsApp, o LinkedIn e o Facebook esperam (1200 × 630), com a marca e a explicação do que é o PRISMA. Depois reduzi o peso dela pela metade sem perder nada da qualidade — de 33 kB para **15 kB**.

### Todos os livros com o mesmo título

> **IMAGINE ASSIM**
>
> Uma estante com 25 livros diferentes, e *todos* com o mesmo título escrito na lombada. Para achar um, você tem que abrir cada um.

Era o caso das abas do navegador. As 25 telas do site tinham o mesmo nome na abinha. O gestor, que abre vários currículos ao mesmo tempo para comparar, ficava com uma fileira de abas idênticas — impossível saber qual era qual.

**O que eu fiz:** cada tela ganhou nome próprio ("Minhas candidaturas", "Publicar vaga", "Currículo do aluno"...). E as telas públicas ganharam também um resumo de uma frase, que é o textinho que aparece embaixo do título nos resultados do Google. Antes não existia resumo nenhum, então o Google inventava um pegando a primeira frase que encontrava na tela — que era "Selecione seu tipo de usuário".

Um detalhe pequeno e engraçado: o site estava declarado como sendo **em inglês**, apesar de todo o texto estar em português. Quem usa leitor de tela ouvia as palavras portuguesas pronunciadas com sotaque de inglês. Corrigido.

## 7 Falando com o Google: o aviso e o mapa

> **IMAGINE ASSIM**
>
> O Google é um bibliotecário que passa no seu prédio para anotar o que tem lá dentro e depois indicar às pessoas. Ele precisa de duas coisas na portaria: um **aviso** dizendo em quais salas ele pode entrar, e um **mapa** das salas que valem a visita.

Nenhum dos dois existia. Criei os dois:

- ✓ **O aviso** libera a área pública e pede para o bibliotecário *não* anotar as telas de dentro — as que têm currículo, CPF e endereço de aluno.
- ✓ **O mapa** lista as 5 telas públicas. As telas de dentro ficam de fora de propósito: precisam de senha e não têm nada para o Google mostrar.

> **ATENÇÃO, PARA NÃO CONFUNDIR**
>
> Esse aviso **não é uma tranca**. É só um pedido de boa educação para o bibliotecário. Quem realmente tranca as telas de dentro é a senha e a checagem de permissão do sistema — e essas já estavam funcionando. O aviso só evita que os endereços apareçam na busca.

## 8 As trancas: segurança

O site tem duas metades que moram em servidores diferentes: a **parte de dentro** (que guarda os dados) e a **parte de fora** (as telas que você vê). Elas precisam de trancas separadas.

### A parte de dentro

Rodei a verificação de segurança oficial que já vem com a ferramenta usada no projeto. Resultado: **nenhum problema**. E não confiei de primeira — rodei de novo, de propósito, com uma configuração insegura, para ver se a verificação realmente reclamava. Ela reclamou de 5 coisas. Ou seja: o resultado limpo era de verdade, não estava desligado.

### A parte de fora

> **IMAGINE ASSIM**
>
> Alguém mal-intencionado pode pegar o seu site e colocá-lo dentro de uma *moldura* num site falso, com um botão invisível por cima. A pessoa acha que está clicando em "Confirmar cadastro" no PRISMA, mas está clicando em outra coisa.

Escrevi um arquivo de trancas para a parte de fora. Ele diz, entre outras coisas: o site não pode ser colocado dentro de moldura nenhuma; só pode buscar informação nos endereços que a gente autorizou; e o navegador deve sempre usar o cadeado, mesmo que a pessoa digite o endereço sem ele. Também desliguei o acesso a câmera, microfone e localização — o site não usa nada disso, e desligar significa que nem um invasor conseguiria pedir.

### A tranca que era obrigatória e faltava

> **ISTO IA QUEBRAR NO PRIMEIRO DIA**
>
> O PRISMA é um site de um único arquivo, que se transforma por dentro conforme você navega. Se a pessoa digitar `/login` **direto** na barra do navegador — ou apertar F5, ou abrir um link salvo — o servidor vai procurar um arquivo chamado "login", não vai achar, e vai mostrar a **página de erro da hospedagem**. Nem a nossa tela 404 bonitinha.
>
> Escrevi a regra que resolve: "para qualquer endereço, entregue o arquivo único e deixe o site decidir o que mostrar".

## 9 Os biscoitinhos e a etiqueta

### Cookies: boa notícia

> **IMAGINE ASSIM**
>
> Cookie é um papelzinho que o site guarda no seu navegador. Tem dois tipos. Um é a **senha do guarda-volumes**: sem ele, você tem que provar quem é a cada passo. O outro é o **espião**: fica anotando por onde você andou para depois te mostrar propaganda.

Fui olhar o que o PRISMA guarda. Ele guarda **só o primeiro tipo** — a senha do guarda-volumes, para você não ter que digitar sua senha a cada clique. Nenhum espião.

Isso é uma boa notícia legal: aquele aviso chato de "aceite os cookies" **não é obrigatório** quando o site só usa o tipo necessário. O que a lei pede nesse caso é apenas explicar direito o que é guardado — e isso agora está escrito no pé da página.

### A etiqueta que não existia (rodapé)

> **POR QUE ISSO IMPORTA DE VERDADE**
>
> O PRISMA pede **CPF, endereço e currículo completo** de alunos. E não dizia em lugar nenhum de quem era o site, nem com quem falar. Uma tela que pede CPF sem se identificar tem exatamente a cara de um site de golpe — mesmo sendo honesta.
>
> E não é só aparência: a Lei Geral de Proteção de Dados (a LGPD) **obriga** a informar quem cuida dos dados e um jeito de entrar em contato.

Criei o rodapé com três blocos: quem é a instituição (nome, CNPJ, endereço); como falar com ela (e-mail, telefone, WhatsApp); e o que é feito com os dados, com o contato de quem responde por eles. Ele aparece só nas telas públicas — nas telas de dentro, cada área já tem o layout dela.

**Só que ele está com os dados em branco**, marcados como `[PREENCHER]`. Isso foi de propósito: colocar um nome inventado seria pior que não colocar nada, porque pareceria verdadeiro. Enquanto estiver em branco, aparece um aviso vermelho na tela dos programadores — e esse aviso desaparece automaticamente na versão que vai para o público.

## 10 O resumo de tudo

| # | Item da sua lista | Situação | Em uma frase |
|---|---|---|---|
| 1 | Abrir no celular | **Feito** | 4 telas não cabiam no celular; agora todas cabem |

| # | Item da sua lista | Situação | Em uma frase |
|---|---|---|---|
| 2 | Testar formulário | **Feito** | Enter não enviava em nenhuma tela; agora envia |
| 3 | Clicar no WhatsApp | **Falta você** | Estrutura pronta, falta o número |
| 4 | Página 404 | **Feito** | Tela de verdade, no lugar do empurrão silencioso |
| 5 | Domínio próprio | **Falta você** | Registrar e rodar um comando que eu deixei pronto |
| 6 | HTTPS (cadeado) | **Metade** | Código pronto; falta o certificado no servidor |
| 7 | PageSpeed | **Feito** | 59% mais leve; falta medir com o site no ar |
| 8 | Comprimir imagens | **Feito** | Não havia imagem nenhuma; a que criei ficou 56% menor |
| 9 | Favicon | **Feito** | Era o crachá de outra empresa |
| 10 | Imagem de compartilhamento | **Feito** | Link no WhatsApp saía sem foto nem título |
| 11 | Sitemap (o mapa) | **Feito** | Não existia |
| 12 | Google Analytics | **Decisão sua** | Muda a regra dos cookies — explico na página seguinte |
| 13 | Cookies | **Feito** | Só o necessário; aviso obrigatório não é preciso |
| 14 | Dados no rodapé | **Falta você** | Rodapé criado, dados em branco |
| 15 | Robots (o aviso) | **Feito** | Não existia |
| 16 | Títulos | **Feito** | 25 telas tinham o mesmo nome de aba |
| 17 | Resumos para o Google | **Feito** | Nenhuma tela tinha resumo |
| 18 | Clicar nos links | **Feito** | 24 links conferidos, nenhum quebrado |
| 19 | Segurança | **Feito** | Verificação oficial limpa + trancas na parte de fora |

**EM NÚMEROS**

**7 itens** estavam quebrados ou faltando de um jeito que só apareceria *depois* de publicar — quando já teria gente tentando usar. **4 itens** dependem de informação que só você tem. Os outros **8** estavam certos e eu confirmei um por um.

## 11 O que só você pode fazer

São quatro coisas, em ordem. A primeira destrava as outras.

- ☐ **1. Registrar o endereço do site (o domínio).** Algo como `prisma.suaescola.br`. Depois de registrar, você roda **um comando** que eu deixei pronto e ele espalha o endereço nos 15 lugares onde ele precisa aparecer:

  ```
  npm run definir-dominio prisma.suaescola.br
  ```

  Fiz esse comando porque trocar à mão em 15 lugares quase garante esquecer um — e cada esquecimento quebra uma coisa diferente, silenciosamente. Um deles, se ficar errado, **impede o login de funcionar**.

- ☐ **2. Preencher os dados do rodapé.** É um arquivo só. Nome da instituição, CNPJ, endereço, e-mail de suporte, e o e-mail de quem responde pelos dados dos alunos (esse é o que a LGPD exige). Se você colocar também o número do WhatsApp, o link aparece sozinho — e isso resolve o item 3 da sua lista.

  Depois de preencher, vale abrir no celular e conferir se o link do WhatsApp abre a conversa certa.

- ☐ **3. Ligar o cadeado (HTTPS) na hospedagem.** O código já está pronto. Falta pedir o certificado — que é de graça, e nos serviços mais comuns de hospedagem é automático. Depois de publicar, dá para conferir a nota num site de teste; a meta é **A**.

- ☐ **4. Decidir sobre o Google Analytics.** Não instalei de propósito, e vale você decidir com calma, porque *muda a situação legal do site*.

  Hoje o PRISMA não tem nenhum cookie espião. No momento em que o Analytics entra, passa a ser **obrigatório** aquele aviso de cookies — e não um aviso qualquer: um que *bloqueie* o rastreamento até a pessoa clicar em "aceito". Num sistema que guarda CPF e currículo de aluno, isso merece uma escolha consciente:

  - ☐ **Se você quer saber quantas pessoas usam o sistema**, o próprio PRISMA já tem esse número por dentro (cadastros, logins, candidaturas) — sem cookie nenhum e sem mandar dado de aluno para fora.
  - ☐ **Se você quer saber de onde vêm os visitantes** (para medir divulgação), aí o Analytics faz sentido. Me avise e eu instalo junto com o aviso de cookies funcionando de verdade.

## 12 Como eu sei que não quebrei nada

Mexer em código sempre tem risco de consertar uma coisa e quebrar outra. Então, no fim, rodei tudo de novo:

- ✓ **80 testes automáticos** da parte de dentro do sistema (cadastro, confirmação de conta, login, troca de senha, permissões): **80 passaram, 0 falharam**.

- ✓ **18 verificações no navegador** (celular, formulário, 404, crachá, imagem de compartilhamento, títulos, links): **0 falhas**.
- ✓ **Verificação oficial de segurança**: nenhum problema.
- ✓ **Verificação de erros de escrita no código** e montagem da versão final: tudo limpo.

E deixei essas 18 verificações do navegador guardadas dentro do projeto, em `tests/checklist_publicacao.py`, para você ou qualquer pessoa da equipe rodar de novo quando quiser — antes de cada publicação, por exemplo. Ele avisa na hora se alguma coisa voltou a quebrar.

**SE VOCÊ LEMBRAR DE UMA COISA SÓ DESTE DOCUMENTO**

Lembre do **endereço errado no captcha**. Aquele `127.0.0.1` significava "aqui, nesta máquina". Se o site tivesse ido para o ar como estava, **nenhum aluno conseguiria criar uma conta** — e a tela não daria nenhuma pista do motivo. Era o tipo de problema que funciona perfeitamente no computador de quem programou e só aparece quando é gente de verdade tentando usar.

---

Documento gerado em 6 de setembro de 2026, sobre a branch `teste-integracao-front`.
A versão técnica deste mesmo relatório, com nomes de arquivo, números e comandos, está em `docs/CHECKLIST_PUBLICACAO.md`.